

Example Exam

Problem Statement

A company designs small electronic circuits. Each circuit contains several pins that must be connected using wires. The goal is to connect all pins while minimizing the total amount of wire used.

The circuit contains the following pins:

A, B, C, D, E, F

The possible wires between pins and their lengths are:

Wire	Lenght
A-B	4
A-C	2
A-D	7
B-C	1
B-E	5
C-D	3
C-E	8
D-E	2
D-F	6
E-F	4

The company wants to obtain a circuit that connects all pins with the minimum total wire length.

Question 1

Describe the data structures required to model this problem. Define the relevant variables and provide a graphical representation of those structures.

Student Response

This problem should be modeled using a weighted undirected graph.

Each pin in the circuit is represented as a vertex of the graph, and each possible wire between two pins is represented as an edge. Since each wire has a length, the edges must be weighted.

The set of vertices is:

$V = \{A, B, C, D, E, F\}$

The set of weighted edges is:

$E = \{$

(A, B, 4),

(A, C, 2),

(A, D, 7),

(B, C, 1),

(B, E, 5),

(C, D, 3),

(C, E, 8),

(D, E, 2),

(D, F, 6),

(E, F, 4)

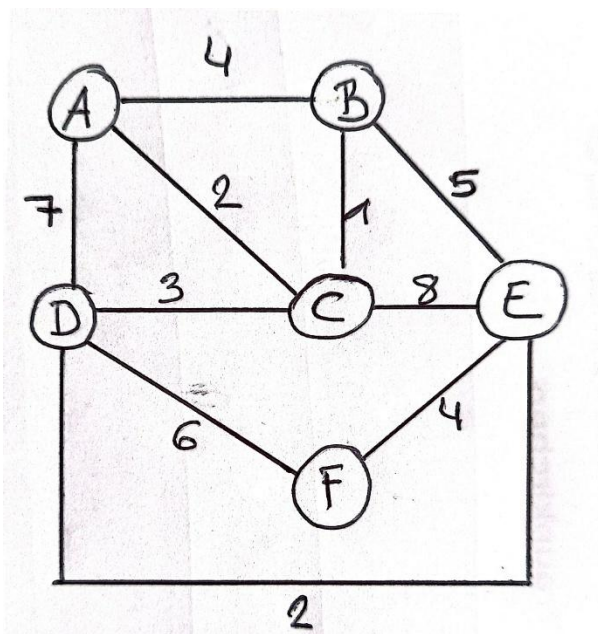
$\}$

A suitable data structure would be an adjacency list, where each vertex stores its neighboring vertices and the corresponding edge weights:

```
graph = {  
  A: [(B, 4), (C, 2), (D, 7)],  
  B: [(A, 4), (C, 1), (E, 5)],  
  C: [(A, 2), (B, 1), (D, 3), (E, 8)],  
  D: [(A, 7), (C, 3), (E, 2), (F, 6)],  
  E: [(B, 5), (C, 8), (D, 2), (F, 4)],  
  F: [(D, 6), (E, 4)]  
}
```

The same graph could also be represented using an adjacency matrix, where rows and columns represent pins and each cell stores the wire length. If two pins are not directly connected, the value could be infinity or zero, depending on the implementation.

A graphical representation of the graph is:



Question 2

Identify well-known algorithms that solve this optimization problem. Briefly describe each one and justify its suitability for meeting the company's requirements.

Student Response

This problem can be solved using different graph algorithms. One possible algorithm is DFS, because DFS visits all the nodes of a graph. Since the company wants all pins to be connected, DFS is useful because it checks whether all pins can be reached.

Another possible algorithm is BFS, which also visits all the nodes. BFS may be better than DFS because it explores the closest nodes first, so it can probably find a short connection between pins.

Prim's algorithm can also be used. It starts from one node and keeps adding nearby nodes until the graph is connected. Kruskal's algorithm is another option because it orders the edges by their weight and adds them to the solution.

I think both Prim and Kruskal have complexity $O(n)$, because each node is visited once. Therefore, they are suitable for the company because they are fast and can connect all the pins.

Cycles should be avoided because they would add unnecessary wires, although BFS and DFS can also avoid cycles if we mark visited nodes.

Question 3

Implement an algorithm that, given the optimized circuit obtained in the previous section, identifies the two pins whose distance, defined as the length of the shortest path between them, is maximum. The algorithm must return that distance and the intermediate pins forming the shortest path between the corresponding pair of pins. The code must include comments explaining how it works. Show an example of how to call the algorithm from a main program, specifying the initial parameter values.

Student Response

The goal is to find the pair of pins whose shortest-path distance is the largest. Therefore, we must compute the shortest path between every pair of pins and then select the maximum among those distances.

```
Function FindFarthestPins (Graph G)
    maxDistance = -infinity
    farthestPin1 = null
    farthestPin2 = null
    farthestPath = empty list

    For each pin u in G do:
        distances, previous = Dijkstra (G, u)
        For each pin v in G do:
            If u ≠ v and distances[v] > maxDistance then:
                maxDistance = distances[v]
                farthestPin1 = u
                farthestPin2 = v
                farthestPath = ReconstructPath (previous, u, v)
            End If
        End For
    End For

    intermediatePins = farthestPath without the first and last
    Return farthestPin1, farthestPin2, maxDistance, intermediatePins
End Function
```

The algorithm works by doing this:

1. Pick one pin as the starting pin.
2. Calculate the shortest distance from that pin to every other pin.
3. Repeat that for every pin.
4. Keep the pair with the largest shortest-path distance.
5. Reconstruct the path between those two pins.